

help me write a python script to access this website. add a check to ensure the website is accessible. keep it simple and concise. explain clearly and concisely what each line of your code does. output in modular/jupyter notebook format. website: "[Project Experts - proSapient]
([https://platform.prosapient.com/client/projects/ef3ae048-29ac-46c6-9b36-66a2a3a9fd60/experts/c2ce1c89-6914-44ea-97fd-0f86fc484d4b?ga=0q13m5l4yLRCxblbSdEz-Ohz2bQKsG7i7ixmKnxgVGtcjVFLLCHBZaUNkUHhA3Lm&utm_campaign=Expert list external link&utm_medium=link&utm_source=bio_email](https://platform.prosapient.com/client/projects/ef3ae048-29ac-46c6-9b36-66a2a3a9fd60/experts/c2ce1c89-6914-44ea-97fd-0f86fc484d4b?ga=0q13m5l4yLRCxblbSdEz-Ohz2bQKsG7i7ixmKnxgVGtcjVFLLCHBZaUNkUHhA3Lm&utm_campaign=Expert%20list%20external%20link&utm_medium=link&utm_source=bio_email))"

Here's a simple, modular Python script to access that URL and check if it's accessible:

python

Cell 1 – Imports

```
import requests # Third-party library for making HTTP requests
```

python

Cell 2 – Configuration

```
URL = "https://platform.prosapient.com/client/projects/ef3ae048-29ac-46c6-9b36-66a2a3a9fd60/experts/c2ce1c89-6914-44ea-97fd-0f86fc484d4b?ga=0q13m5l4yLRCxblbSdEz-Ohz2bQKsG7i7ixmKnxgVGtcjVFLLCHBZaUNkUHhA3Lm&utm_campaign=Expert list external link&utm_medium=link&utm_source=bio_email"
```

Stores the target URL as a constant for easy reuse/modification

```
HEADERS = {"User-Agent": "Mozilla/5.0"}
```

Mimics a real browser request; some sites block Python's default

python

Cell 3 – Accessibility check function

```
def check_accessibility(url: str) -> bool:
    """Returns True if the site responds with HTTP 200 OK."""
    try:
        response = requests.head(url, headers=HEADERS, timeout=10)
        # HEAD request fetches only headers (faster than GET – 1
        # timeout=10 prevents hanging indefinitely if the server
        return response.status_code == 200
        # True if accessible, False for any other status code (4
    except requests.exceptions.RequestException as e:
        # Catches all network errors: DNS failure, timeout, SSL
        print(f"Connection error: {e}")
        return False
```

python

Cell 4 – Fetch page content function

```
def fetch_page(url: str) -> str | None:
    """Returns page HTML as a string, or None if the request fails"""
    try:
        response = requests.get(url, headers=HEADERS, timeout=10)
        # GET request retrieves the full page content
        response.raise_for_status()
        # Raises an exception for 4xx/5xx status codes automatically
        return response.text
        # Returns the raw HTML of the page as a string
    except requests.exceptions.RequestException as e:
        print(f"Failed to fetch page: {e}")
        return None
```

python

```
# Cell 5 – Main execution
if check_accessibility(URL):
    print("✅ Site is accessible")
    html = fetch_page(URL)
    if html:
        print(f"✅ Page fetched successfully ({len(html):,} characters)")
        print("\n--- First 500 characters of HTML ---")
        print(html[:500]) # Preview a snippet so you're not flooded
    else:
        print("❌ Site is not accessible – check the URL or your network")
```

A few things to be aware of:

- **This URL requires authentication.** The `ga=...` token in the URL is a temporary access token from proSapient. It may expire, so if you get a `403` or are redirected to a login page, the token has likely expired and you'll need a fresh link.
- **Install `requests`** if you haven't: `pip install requests`
- **To parse the HTML** (extract specific data), add `pip install beautifulsoup4` and use `BeautifulSoup(html, "html.parser")` in a new cell.

i got this error: "Connection error:
 HTTPSConnectionPool(host='platform.prosapient.com',
 port=443): Max retries exceeded with url:
 /client/projects/ef3ae048-29ac-46c6-9b36-
 66a2a3a9fd60/experts/c2ce1c89-6914-44ea-97fd-
 0f86fc484d4b?ga=0q13m5l4yLRCxblbSdEz-
 Oh72bOKsG7i7ixmKnxgVGtcivFI I CHR7aUJNkUJHhA3I m

Show more

This is an **SSL certificate verification error** — your Python environment can't verify the site's certificate chain. This is common on macOS or in corporate/VPN environments. Here are two fixes, from safest to quickest:

Fix 1 — Proper fix: provide the CA bundle (recommended)

python

```
# Cell 2 – Configuration (updated)
import certifi # Provides an up-to-date Mozilla CA certificate

URL = "https://platform.prosapiant.com/client/projects/..."
HEADERS = {"User-Agent": "Mozilla/5.0"}
CA_BUNDLE = certifi.where() # Returns the local path to the trust bundle
```

python

```
# Cell 3 – Accessibility check (updated)
def check_accessibility(url: str) -> bool:
    try:
        response = requests.head(url, headers=HEADERS, timeout=10)
        # verify=CA_BUNDLE tells requests to use certifi's certificate bundle
        return response.status_code == 200
    except requests.exceptions.RequestException as e:
        print(f"Connection error: {e}")
        return False
```

python

```
# Cell 4 – Fetch page (updated)
def fetch_page(url: str) -> str | None:
    try:
        response = requests.get(url, headers=HEADERS, timeout=10)
        response.raise_for_status()
        return response.text
    except requests.exceptions.RequestException as e:
        print(f"Failed to fetch page: {e}")
        return None
```

Install certifi first:

```
bash

pip install certifi
```

Fix 2 — Quick fix: disable SSL verification (not recommended for production)

```
python

# Replace verify=CA_BUNDLE with verify=False in both functions
# Also add this to suppress the warning it generates:
import urllib3
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
# Without this, every request prints a noisy security warning to the console

response = requests.head(url, headers=HEADERS, timeout=10, verify=False)
# verify=False skips certificate checking entirely – fine for quick testing
# but leaves you open to man-in-the-middle attacks in production
```

Which to use?

Situation	Fix
macOS / outdated certs	Fix 1 (certifi)
Corporate network / VPN / proxy	Fix 1 first; if still failing, Fix 2 temporarily
Just testing quickly	Fix 2 is fine

Most likely cause: You're on macOS and Python's cert store is out of date — running `/Applications/Python 3.x/Install Certificates.command` also permanently fixes this system-wide.

i am working on a company laptop. give the step by step process in clear, concise, and simple terms in how to do the proper fix

Here's the step-by-step process:

Step 1 — Check if certifi is already installed

Open your terminal/command prompt and run:

```
bash  
  
pip show certifi
```

- If you see version info → skip to Step 3
 - If you see "not found" → go to Step 2
-

Step 2 — Install certifi

```
bash  
  
pip install certifi
```

Step 3 — Find your company's CA certificate

Your company laptop likely uses a **custom corporate certificate** to inspect HTTPS traffic (common with VPNs/proxies). You need to export it:

On Windows:

1. Press **Win + R**, type **certmgr.msc**, hit Enter
2. Go to **Trusted Root Certification Authorities → Certificates**
3. Find your company's certificate (usually named after your company or IT provider)
4. Right-click it → **All Tasks → Export**
5. Choose format: **Base-64 encoded X.509 (.CER)**
6. Save it somewhere easy, e.g. **C:\certs\company.cer**

On Mac:

1. Open **Keychain Access** (search it in Spotlight)
 2. Go to **System → Certificates**
 3. Find your company certificate → right-click → **Export**
 4. Save as **.pem** format
-

Step 4 — Append the corporate cert to certifi's bundle

Run this in your terminal, replacing the path with where you saved the cert:

```
bash

# Windows
type C:\certs\company.cer >> "%USERPROFILE%\AppData\Local\Programs\Python\Python39-32\Scripts\certifi.exe"

# Mac/Linux
cat /path/to/company.cer >> $(python -c "import certifi; print(certifi.where())")
```

This adds your company cert to the trusted list that `certifi` uses.

Step 5 — Update your script's Cell 2

```
python

import certifi
CA_BUNDLE = certifi.where() # Now includes your company certifi
```

Step 6 — Re-run your notebook

If it works → you're done. If you still get an SSL error, it likely means your company's IT policy is blocking external requests entirely, and you'll need to raise a ticket with your IT team to whitelist

`platform.prosapiant.com`.

explain what step 4 code does

Windows version:

```
bash
```



```
type C:\certs\company.cer >> "%USERPROFILE%\AppData\Local\Programs
```

- `type` — Windows command that reads and prints a file's contents
- `C:\certs\company.cer` — the company certificate you exported in Step 3
- `>>` — appends the output to the end of the target file (rather than overwriting it)
- `%USERPROFILE%\...certifi\cacert.pem` — the certifi certificate bundle file; this is the master list of trusted certificates that Python uses to verify websites

In plain English: *"Read my company certificate and add it to the bottom of Python's trusted certificates list."*

Mac/Linux version:

```
bash
```

```
cat /path/to/company.cer >> $(python -c "import certifi; print(c
```

- `cat` — Mac/Linux equivalent of `type`; reads and prints a file's contents
- `/path/to/company.cer` — the company certificate you exported in Step 3
- `>>` — same as above; appends rather than overwrites
- `$(...)` — runs the command inside and substitutes its output inline
- `python -c "import certifi; print(certifi.where())"` — a tiny one-line Python script that prints the exact file path of certifi's certificate bundle on your machine

In plain English: *"Read my company certificate, find where certifi's trusted list lives on this machine, and add the certificate to the bottom of it."*

Why this works: proSapient's SSL certificate is signed by your company's internal certificate authority. Since that authority isn't in the default trusted list, Python rejects it. By appending it, you're telling Python *"trust this authority too"* — so verification passes.

i got the error "The system cannot find the path specified."

This means the path to certifi's `cacert.pem` file is wrong on your machine. Let's find the correct path first.

Step 1 — Find the exact path on your machine

Run this in your terminal:

```
bash  
  
python -c "import certifi; print(certifi.where())"
```

This will print the exact path, something like:

```
C:\Users\YourName\AppData\Local\Programs\Python\Python311\Lib\site-packages\certifi\cacert.pem
```



Step 2 — Run the append command with the correct path

Copy the path printed above and use it in the command:

```
bash
```

```
type C:\certs\company.cer >> "PASTE_THE_PATH_HERE"
```

For example:

```
bash
```

```
type C:\certs\company.cer >> "C:\Users\YourName\AppData\Local\Pr
```

A horizontal scrollbar is located below the command line, indicating that the text in the terminal window is longer than the visible area.

Step 3 — Verify it worked

Run this to confirm the file was appended to:

```
bash
```

```
python -c "import certifi; print(certifi.where())"
```

Then open that file in Notepad and scroll to the very bottom — you should see your company certificate appended there.

The key issue was simply that the path in Step 4 was a **generic example** — the actual path varies depending on your Windows username and Python version, which is why finding it first with `certifi.where()` is the safest approach.

i got Python was not found; run without arguments to install from the Microsoft Store, or disable this shortcut

from Settings > Apps > Advanced app settings > App execution aliases.

This means Python isn't recognised in your terminal. Let's find where it's installed first.

Step 1 — Try `python3` instead

```
bash
```

```
python3 -c "import certifi; print(certifi.where())"
```

If that works, use `python3` everywhere instead of `python`. If not, go to Step 2.

Step 2 — Try `py` instead

```
bash
```

```
py -c "import certifi; print(certifi.where())"
```

Windows sometimes installs Python under the `py` launcher. If that works, you're done — copy the path and go back to the append command.

Step 3 — If both fail, find where Python is installed

```
bash
```

```
where python
```

```
where python3
```

```
where py
```

This prints the full path to Python on your machine, e.g:

```
C:\Users\YourName\AppData\Local\Programs\Python\Python311\python
```



Step 4 — If none of the above work

You're likely using a **virtual environment or Anaconda**. Try:

```
bash

# If using Anaconda/conda
conda run python -c "import certifi; print(certifi.where())"

# If using a virtual environment, activate it first
.\venv\Scripts\activate
python -c "import certifi; print(certifi.where())"
```

Tell me which step works and I'll guide you from there. Also let me know what you used to run your Jupyter notebook (Anaconda, VS Code, plain Python, etc.) — that'll help narrow it down quickly.

step 2 worked

Great! Now run this to get the exact certifi path:

Step 1 — Get the exact path

```
bash
```

```
py -c "import certifi; print(certifi.where())"
```

It will print something like:

```
C:\Users\YourName\AppData\Local\Programs\Python\Python311\Lib\site-packages\certifi\cacert.pem
```



Step 2 — Append your company certificate

Copy the path printed above and run:

```
bash
```

```
type C:\certs\company.cer >> "PASTE_THE_PATH_HERE"
```

Step 3 — Confirm it worked

Open the file printed in Step 1 in Notepad and scroll to the very bottom — you should see your company certificate text starting with:

```
-----BEGIN CERTIFICATE-----
```

Then go back to your Jupyter notebook and re-run it. Let me know what happens!

Also, for future terminal commands in this guide, use `py` instead of `python` — that's the correct command for your machine.

i got access is denied

This is a **permissions issue** — your company laptop is blocking writes to that folder. This is common on corporate machines. Here are your options:

Option 1 — Run terminal as Administrator

1. Press **Win + S**, search for **Command Prompt**
2. Right-click it → **Run as administrator**
3. Re-run the append command:

```
bash
```

```
type C:\certs\company.cer >> "PASTE_THE_PATH_HERE"
```

Option 2 — If Option 1 is blocked by IT, use **verify=False** as a workaround

This is the quickest practical solution on a locked-down corporate laptop. Update your script:

```
python
```

```
# Cell 1 - Imports
import requests
import urllib3
```

```
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarn
# Suppresses the SSL warning that appears when verify=False is u
```

python

Cell 3 – Accessibility check

```
def check_accessibility(url: str) -> bool:
    try:
        response = requests.head(url, headers=HEADERS, timeout=10)
        return response.status_code == 200
    except requests.exceptions.RequestException as e:
        print(f"Connection error: {e}")
        return False
```

python

Cell 4 – Fetch page

```
def fetch_page(url: str) -> str | None:
    try:
        response = requests.get(url, headers=HEADERS, timeout=10)
        response.raise_for_status()
        return response.text
    except requests.exceptions.RequestException as e:
        print(f"Failed to fetch page: {e}")
        return None
```

Which to try:

Situation	Action
You have admin rights	Try Option 1
Admin is blocked by IT	Use Option 2

Situation	Action
Neither works	IT has blocked the site entirely — raise a ticket

Option 2 is safe enough for **internal company tools** like proSapient — the risk is low since you're not transmitting sensitive credentials over an unknown network.

explain in simple terms - what does option 2 do?

Normal HTTPS connection (without `verify=False`):

Python makes a request to the website and asks *"can I see your certificate so I can verify you are who you say you are?"* The website shows its certificate, Python checks it against its trusted list, and only connects if it passes.

Your company laptop **intercepts** this check and inserts its own corporate certificate — which Python doesn't recognise — so Python refuses to connect and throws the SSL error.

With `verify=False`:

Python makes a request to the website but this time **skips the certificate check entirely** — it just connects without asking for ID. It no longer cares whether the certificate is trusted or not.

The risk in simple terms:

Normally the certificate check ensures you're talking to the **real** website and not an impostor. Skipping it means Python can't

confirm this. However in your case:

- You're on a **corporate network** that already monitors traffic
- The website URL came directly **from proSapient via email**
- You're not entering passwords or payment details via this script

So the practical risk is **very low** in this specific situation.

all worked. i now put it all in one python file and got this error when running: "

```
[Running] python -u "c:\Users\ND126HF\00 EN
Workflow Automation\access_test_for_PA.py"
Traceback (most recent call last):
  File "c:\Users\ND126HF\00 EN Workflow
```

Show more

Simple fix — your terminal doesn't support emoji characters.
Replace the emojis with plain text:

```
python

# Cell 5 – Main execution
if check_accessibility(URL):
    print("SUCCESS: Site is accessible")
    html = fetch_page(URL)
    if html:
        print(f"SUCCESS: Page fetched successfully ({len(html):
        print("\n--- First 500 characters of HTML ---")
        print(html[:500])
```

```
else:
    print("FAILED: Site is not accessible – check the URL or you
```

Or alternatively, force UTF-8 encoding at the top of your script:

```
python

# Add this as the very first lines of your script
import sys
import io
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')
# Forces the terminal to use UTF-8 encoding, which supports emoji
```

Which to use:

Option	When
Replace emojis with plain text	Quickest fix, always works
Force UTF-8	If you want to keep emojis throughout your script

i now got this "

```
SUCCESS: Site is accessible
SUCCESS: Page fetched successfully (4,712
characters)

--- First 500 characters of HTML ---
```

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <link rel="shortcut icon"
href="/favicon.ico" />
    <meta name="viewport" content="width=device-
width, initial-scale=1, shrink-to-fit=no" />
    <meta name="theme-color" content="#000000"
/>
    <meta name="CSRF_TOKEN"
content="OXEkKCU7AxMqYmJ4EVYbHg0rJSoJHSAUp7Po1R4
k1--Ur2LDDCaGZ0TE" />
    <link
href="https://fonts.googleapis.com/css?
family=Roboto:300,400,500,700&display=swap&subse
t=cyrillic" rel="stylesheet" />
    <t
```

"

Show less

Your script is working perfectly! Here's what the output tells you:

SUCCESS: Site is accessible The accessibility check passed — the site responded with HTTP 200 OK.

SUCCESS: Page fetched successfully (4,712 characters) The full page HTML was retrieved — 4,712 characters worth of content.

What the HTML tells you:

- `<!DOCTYPE html>` — it's a standard web page
 - `lang="en"` — English language site
 - `CSRF_TOKEN` — the site uses **security tokens** to protect against unauthorised requests, meaning the real data is loaded **after** login, not in this initial HTML
 - `fonts.googleapis.com` — it loads fonts from Google, confirming it's a live page
-

The key limitation now:

The 4,712 characters you got is just the **empty page shell** — the actual expert data (names, profiles, etc.) is loaded dynamically after authentication. To access that you'd need to either:

Approach	Complexity
Use <code>selenium</code> to log in and interact with the page like a real browser	Medium
Use proSapient's API directly if they offer one	Low — ask proSapient
Inspect the network tab in your browser to find the API calls the page makes	Medium

What are you ultimately trying to do with this page? That'll help determine the best next step.

i'd like to first try to use selenium -provide a simple, clear, and concise step by step process

Step 1 — Install Selenium and the Chrome driver manager

```
bash
```

```
pip install selenium webdriver-manager
```

- `selenium` — controls the browser programmatically
 - `webdriver-manager` — automatically downloads the correct Chrome driver for your machine
-

Step 2 — Check Chrome is installed on your machine

Open Chrome and go to:

```
chrome://version
```

Note the version number (e.g. `124.0.6367.82`) — the driver manager will match this automatically.

Step 3 — Add this to your Python file

```
python
```

```
# Cell 1 — Imports
```

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from webdriver_manager.chrome import ChromeDriverManager
import time
```



python

Cell 2 – Configuration

```
URL = "https://platform.prosapiant.com/client/projects/ef3ae048"
```

python

Cell 3 – Browser setup

```
def create_driver():
    options = webdriver.ChromeOptions()
    options.add_argument("--ignore-certificate-errors")
    # Bypasses the SSL certificate issue we encountered earlier
    options.add_argument("--start-maximized")
    # Opens browser in full screen

    driver = webdriver.Chrome(
        service=Service(ChromeDriverManager().install()),
        options=options
    )
    # Automatically downloads and sets up the correct Chrome driver
    return driver
```

python

Cell 4 – Fetch page content

```
def fetch_with_selenium(url: str) -> str | None:
    driver = create_driver()
    try:
        driver.get(url)
        # Opens the URL in Chrome

        WebDriverWait(driver, 10).until(
            EC.presence_of_element_located((By.TAG_NAME, "body"))
        )
        # Waits up to 10 seconds for the page body to load before
```

```
time.sleep(3)
# Extra 3 second wait for dynamic content (JavaScript)

content = driver.find_element(By.TAG_NAME, "body").text
# Extracts all visible text from the page body

return content
except Exception as e:
    print(f"Error: {e}")
    return None
finally:
    driver.quit()
    # Always closes the browser, even if an error occurs
```

python

```
# Cell 5 – Main execution
content = fetch_with_selenium(URL)
if content:
    print(f"SUCCESS: Page fetched ({len(content):,} characters)")
    print("\n--- First 500 characters ---")
    print(content[:500])
else:
    print("FAILED: Could not fetch page content")
```

What to expect when you run it:

1. A real Chrome browser window will pop open
2. It navigates to the URL automatically
3. Waits for the page to load
4. Extracts the visible text
5. Closes the browser

Run it and share the output — if the page requires a login step we'll handle that next.

```
got this error: "SSLCertVerificationError
Traceback (most recent call last)
File
c:\Users\ND126HF\AppData\Local\Programs\Python\Python313\Lib\site-packages\urllib3\connectionpool.py:464,
in HTTPConnectionPool._make_request(self, conn,
method, url, body, headers, retries, timeout, chunked,
```

Show more

This error is coming from **webdriver-manager** trying to download **the Chrome driver** — it's hitting the same SSL/corporate network issue as before. Let's fix it:

Option 1 — Tell webdriver-manager to skip SSL verification

Add this at the very top of your script, before all imports:

```
python

# Cell 0 – Environment setup (must be first)
import os
os.environ['WDM_SSL_VERIFY'] = '0'
# Tells webdriver-manager to skip SSL verification when downloading
```

Option 2 — If Option 1 doesn't work, manually download the driver

1. Open Chrome and go to `chrome://version` — note your exact version number (e.g. `124.0.6367.82`)
2. Go to: <https://googlechromelabs.github.io/chrome-for-testing/>
3. Download the `chromedriver` that matches your Chrome version and **Windows 64-bit**
4. Unzip it and save `chromedriver.exe` somewhere simple, e.g. `C:\chromedriver\chromedriver.exe`
5. Update Cell 3 in your script to point to it directly:

```
python

# Cell 3 – Browser setup (manual driver path)
def create_driver():
    options = webdriver.ChromeOptions()
    options.add_argument("--ignore-certificate-errors")
    options.add_argument("--start-maximized")

    driver = webdriver.Chrome(
        service=Service("C:\\chromedriver\\chromedriver.exe"),
        # Points directly to the manually downloaded driver – no need for system path
        options=options
    )
    return driver
```

Try Option 1 first as it's the quickest. Let me know what happens!

this worked. now i want all the expert names, their roles, relevant company, segment (e.g. customer, competitor, etc), and screener responses if available

First, let's inspect what the page actually looks like to Selenium.
Run this to see the full page content:

```
python

# Cell 5 – Inspect full page content first
content = fetch_with_selenium(URL)
if content:
    print(f"SUCCESS: Page fetched ({len(content):,} characters)")
    print("\n--- FULL PAGE CONTENT ---")
    print(content)
```

Share the full output here — this tells us:

1. Whether the page loaded the expert data or just a login screen
2. The exact text/structure of the data so we can target the right fields
3. Whether screener responses are visible at this level or need a deeper click

This is a necessary first step before writing the extraction code — we need to see what Selenium actually sees before we can parse it accurately.

SUCCESS: Page fetched
(5,681 characters) --- FULL
PAGE CONTENT --- Projects
Calls Autobooking Surveys
Sign In Integrated Workplace
Managemen...

PASTED

full output: ""

The page content is clear and well-structured. Here's the extraction code:

python

Cell 1 – Imports

import os

os.environ['WDM_SSL_VERIFY'] = '0'

from selenium import webdriver

from selenium.webdriver.chrome.service import Service

from selenium.webdriver.common.by import By

from selenium.webdriver.support.ui import WebDriverWait

from selenium.webdriver.support import expected_conditions as EC

import time

import re

python

Cell 2 – Configuration

URL = "https://platform.prosapient.com/client/projects/ef3ae048-

python

Cell 3 – Browser setup

```
def create_driver():
    options = webdriver.ChromeOptions()
    options.add_argument("--ignore-certificate-errors")
    options.add_argument("--start-maximized")
    driver = webdriver.Chrome(
        service=Service("C:\\chromedriver\\chromedriver.exe"),
        options=options
    )
    return driver
```

python

Cell 4 – Fetch full page content (with Load More clicking)

```
def fetch_full_content(url: str) -> str:
    driver = create_driver()
    try:
        driver.get(url)
        WebDriverWait(driver, 10).until(
            EC.presence_of_element_located((By.TAG_NAME, "body"))
        )
        time.sleep(3)

        # Keep clicking "Load more" until all experts are loaded
        while True:
            try:
                load_more = driver.find_element(By.XPATH, "//button")
                # Finds the "Load more" button by its text
                load_more.click()
                time.sleep(2)
                # Waits for new experts to load after each click
            except:
                break
            # Exits loop when no more "Load more" button is found

    content = driver.find_element(By.TAG_NAME, "body").text
```

```

        return content
    except Exception as e:
        print(f"Error: {e}")
        return ""
    finally:
        driver.quit()

```

python

Cell 5 – Parse experts from page content

```

def parse_experts(content: str) -> list:
    experts = []
    lines = content.split('\n')
    # Splits the full page text into individual lines

    i = 0
    while i < len(lines):
        line = lines[i].strip()

        # Detect expert name lines (start with bullet point)
        if line.startswith('•'):
            name = line.replace('•', '').strip()
            # Removes the bullet point and whitespace to get clean name

            # Role/company is the line immediately before the name
            role_company = lines[i-1].strip() if i > 0 else ""

            # Segment and geo are on lines after the name
            geo = lines[i+1].strip() if i+1 < len(lines) else ""
            segment = lines[i+2].strip() if i+2 < len(lines) else ""

            experts.append({
                "name": name,
                "role_company": role_company,
                "geography": geo,
                "segment": segment,
            })
        i += 1

```

```
    })
    i += 1

return experts
```

python

```
# Cell 6 – Display results
content = fetch_full_content(URL)
experts = parse_experts(content)

print(f"Total experts found: {len(experts)}\n")
print(f"{'Name':<45} {'Geography':<20} {'Segment':<15} {'Role/Co
print("-" * 130)
for e in experts:
    print(f"{e['name']:<45} {e['geography']:<20} {e['segment']:<15} {e['role/comp']:<20}")
```

What each cell does in plain English:

- **Cell 4** — Opens the page and keeps clicking "Load more" until all 39 experts are visible
- **Cell 5** — Goes line by line through the text, finds each expert by the  bullet point, then grabs the lines immediately before and after it for role, geography, and segment
- **Cell 6** — Prints everything in a clean table format

Run this and share the output. Screener responses will likely require clicking into each expert's individual profile — we'll tackle that in the next step once this works. ►

```
i got this error for cell 6: " Traceback (most recent call
last) File
c:\Users\ND126HF\AppData\Local\Programs\Python\Pyth
on313\Lib\site-
packages\selenium\webdriver\common\driver_finder.py:
64, in DriverFinder._binary_paths(self) 63 if not
Path(nath) is file(): ---> 64 raise ValueError(f"The path
```

Show more

The chromedriver.exe isn't at `C:\chromedriver\`. Let's find where you saved it:

Step 1 — Find where chromedriver.exe is saved

Open File Explorer and search for `chromedriver.exe` on your machine. Once found, copy the full path (e.g.

`C:\Users\ND126HF\Downloads\chromedriver.exe`).

Step 2 — Update Cell 3 with the correct path

python

Cell 3 – Browser setup

```
def create_driver():
    options = webdriver.ChromeOptions()
    options.add_argument("--ignore-certificate-errors")
    options.add_argument("--start-maximized")
    driver = webdriver.Chrome(
        service=Service("PASTE_YOUR_FULL_PATH_HERE"),
        # Example: "C:\\Users\\ND126HF\\Downloads\\chromedriver
        # Note: use double backslashes \\ in Python paths on Win
```



```
        options=options
    )
    return driver
```

Step 3 — Recommended: move it to a permanent location

To avoid this issue in future, move `chromedriver.exe` to a simple fixed path like:

```
C:\chromedriver\chromedriver.exe
```

Then update Cell 3 to point there. This keeps things clean and consistent.

Share the path you find and I can update the code for you if needed.

the search for chromedriver.exe isn't loading. shall i just install it?

Yes, simplest approach — go back to using **webdriver-manager** to handle it automatically. Just make sure the SSL fix from earlier is in place:

Update Cell 3 to this:

```
python
```

Cell 3 – Browser setup

```
def create_driver():
    options = webdriver.ChromeOptions()
    options.add_argument("--ignore-certificate-errors")
    options.add_argument("--start-maximized")

    driver = webdriver.Chrome(
        service=Service(ChromeDriverManager().install()),
        # webdriver-manager automatically downloads the correct
        options=options
    )
    return driver
```

And make sure Cell 1 has the SSL fix at the very top:

python

Cell 1 – Imports (SSL fix must be first)

```
import os
os.environ['WDM_SSL_VERIFY'] = '0'
# Must be set before any other imports – disables SSL check for

from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from webdriver_manager.chrome import ChromeDriverManager
import time
```

Then re-run all cells top to bottom. This worked for you earlier so it should work again — the only difference now is we've added the "Load more" clicking logic.

what does this code do?

Here's a plain English explanation of the full script, cell by cell:

Cell 1 — Imports Sets up the SSL fix and loads all the tools needed:

- `os.environ['WDM_SSL_VERIFY'] = '0'` — disables SSL check for corporate network
 - `selenium` — controls the Chrome browser
 - `WebDriverWait / EC` — waits for page elements to load before proceeding
 - `ChromeDriverManager` — automatically downloads the correct Chrome driver
 - `time` — used to add waiting pauses
-

Cell 2 — Configuration Just stores the URL as a variable so it's easy to change in one place.

Cell 3 — Browser setup Creates and configures a Chrome browser instance:

- `--ignore-certificate-errors` — bypasses the SSL issue on your corporate network
- `--start-maximized` — opens browser full screen

- `ChromeDriverManager().install()` — auto-downloads the correct driver for your Chrome version
-

Cell 4 — Fetch full page content Opens the URL in Chrome and loads ALL experts:

1. Opens the page and waits for it to load
 2. Looks for a "Load more" button and keeps clicking it until all 39 experts are visible
 3. Extracts all visible text from the page
 4. Closes the browser
-

Cell 5 — Parse experts Goes through the page text line by line and pulls out each expert's details:

- Detects each expert by the  bullet point before their name
 - Grabs the line **before** the bullet for their role and company
 - Grabs the lines **after** the bullet for their geography and segment
-

Cell 6 — Display results Runs everything and prints the results in a clean table showing name, geography, segment, and role/company for all experts.

In one sentence: **it opens the proSapient page in Chrome, loads all experts, reads the text, and organises it into a structured table.**

does it have to be done in chrome? what about
microsoft edge?

No, it doesn't have to be Chrome. Edge works just as well and is **likely already installed on your company laptop** since it comes built-in with Windows. Here's how to switch:

Step 1 — Install the Edge driver manager

```
bash

pip install webdriver-manager
```

Already installed if you did this earlier — no need to reinstall.

Step 2 — Update Cell 1 — swap Chrome imports for Edge

```
python

# Cell 1 – Imports
import os
os.environ['WDM_SSL_VERIFY'] = '0'

from selenium import webdriver
from selenium.webdriver.edge.service import Service
# Edge uses its own Service class instead of Chrome's
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from webdriver_manager.microsoft import EdgeChromiumManager
# Edge uses EdgeChromiumManager instead of ChromeDriverManager
import time
```

Step 3 — Update Cell 3 — use Edge driver

```
python

# Cell 3 – Browser setup
def create_driver():
    options = webdriver.EdgeOptions()
    # EdgeOptions instead of ChromeOptions
    options.add_argument("--ignore-certificate-errors")
    options.add_argument("--start-maximized")

    driver = webdriver.Edge(
        service=Service(EdgeChromiumManager().install()),
        # EdgeChromiumManager automatically downloads the correct
        options=options
    )
    return driver
```



Cells 2, 4, 5, and 6 stay exactly the same — no changes needed there.

Why Edge might actually work better here:

- Already installed on Windows — no need to worry about Chrome being available
- Uses the same underlying engine as Chrome so behaviour is identical
- Corporate IT policies sometimes restrict Chrome but allow Edge

i got this issue when running cell 1: "ImportError: cannot import name 'EdgeChromiumManager' from 'webdriver_manager.microsoft' (c:\Users\ND126HF\AppData\Local\Programs\Python\Python313\Lib\site-packages\webdriver_manager\microsoft.py)"

The class was renamed in a newer version of webdriver-manager.
Simple fix:

Update Cell 1 to this:

```
python

# Cell 1 – Imports
import os
os.environ['WDM_SSL_VERIFY'] = '0'

from selenium import webdriver
from selenium.webdriver.edge.service import Service
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from webdriver_manager.microsoft import EdgeChromiumDriverManager
# Newer versions of webdriver-manager use EdgeChromiumDriverManager
import time
```

And update Cell 3 to match:

```
python
```

```
# Cell 3 – Browser setup
def create_driver():
    options = webdriver.EdgeOptions()
    options.add_argument("--ignore-certificate-errors")
    options.add_argument("--start-maximized")

    driver = webdriver.Edge(
        service=Service(EdgeChromiumDriverManager().install()),
        # Updated to match the new class name
        options=options
    )
    return driver
```

Re-run Cell 1 and Cell 3 then try Cell 6. Let me know what happens!

i also want to obtain each expert's availabilities and screener responses. remember i'm using microsoft edge

First, let's check what's inside an individual expert's profile page.
Run this inspection cell to see what data is available:

```
python

# Cell 7 – Inspect a single expert profile
def inspect_expert_profile(main_url: str) -> None:
    driver = create_driver()
    try:
        # First load the main page
        driver.get(main_url)
        WebDriverWait(driver, 10).until(
            EC.presence_of_element_located((By.TAG_NAME, "body"))
```



```

    )
    time.sleep(3)

    # Find all expert profile links on the page
    expert_links = driver.find_elements(By.XPATH, "//a[contains(@href, '/experts/')]")
    # Looks for all hyperlinks that contain '/experts/' in their href

    if not expert_links:
        print("No expert profile links found")
        return

    # Get the URL of the first expert
    first_expert_url = expert_links[0].get_attribute("href")
    print(f"Opening first expert profile: {first_expert_url}")

    # Navigate to the first expert's profile
    driver.get(first_expert_url)
    WebDriverWait(driver, 10).until(
        EC.presence_of_element_located((By.TAG_NAME, "body"))
    )
    time.sleep(3)

    # Print full profile content
    content = driver.find_element(By.TAG_NAME, "body").text
    print("\n--- EXPERT PROFILE CONTENT ---")
    print(content)

except Exception as e:
    print(f"Error: {e}")
finally:
    driver.quit()

# Run inspection
inspect_expert_profile(URL)

```

What this does:

1. Opens the main experts list page
 2. Finds all clickable expert profile links
 3. Clicks into the **first expert's profile**
 4. Prints everything visible on that profile page
-

Share the full output — this tells us exactly where availability and screener responses sit on the page so we can extract them accurately for all 39 experts.

```
output: "Error: Message: unknown error:
net::ERR_CONNECTION_CLOSED (Session info:
MicrosoftEdge=148.0.3967.54) Stacktrace:
msedgedriver!GetHandleVerifier [0x7ff68bb4e295+e0f5]
msedgedriver!GetHandleVerifier [0x7ff68bb4e2f4+e154]
msedgedriver!GetHandleVerifier
[0x7ff68bec8bca+388a2a] msedgedriver!(No symbol)
```

Show more

ERR_CONNECTION_CLOSED means the site closed the connection when Selenium tried to navigate to an individual expert's profile — likely because the session/token expired or the site detected automated browsing.

Let's try two things:

Step 1 — First check if expert links even exist on the page

Run this simpler diagnostic cell first:

python

Cell 7 – Check what links exist on the page

```
def inspect_links(url: str) -> None:
    driver = create_driver()
    try:
        driver.get(url)
        WebDriverWait(driver, 10).until(
            EC.presence_of_element_located((By.TAG_NAME, "body"))
        )
        time.sleep(3)

        # Find ALL links on the page
        all_links = driver.find_elements(By.TAG_NAME, "a")
        # Gets every hyperlink on the page

        print(f"Total links found: {len(all_links)}\n")
        for link in all_links:
            href = link.get_attribute("href")
            text = link.text.strip()
            if href:
                print(f"Text: {text:<40} URL: {href}")
                # Prints each link's visible text and its URL

    except Exception as e:
        print(f"Error: {e}")
    finally:
        driver.quit()

inspect_links(URL)
```

Share the output — this tells us:

1. Whether individual expert profile links exist at all
2. What URL structure they use
3. Whether the session token in the main URL also needs to be passed to individual profiles

This will determine the best approach before we write the full extraction code.

```
output: "Total links found: 16 Text: URL:
https://platform.prosapiant.com/client Text: Projects
URL: https://platform.prosapiant.com/client/projects
Text: Calls URL:
https://platform.prosapiant.com/client/calls Text:
Autobooking URL:
https://platform.prosapiant.com/client/autobooking Text:
```

Show more

This is very informative. The key finding is that **there are no individual expert profile links** — the expert names are not clickable links. Instead, they're likely opened via a **click event** (a popup or side panel) rather than a separate URL.

Let's verify this:

```
python
```

```
# Cell 7 – Click on the first expert name and capture what appears
def inspect_expert_click(url: str) -> None:
    driver = create_driver()
    try:
        driver.get(url)
        WebDriverWait(driver, 10).until(
            EC.presence_of_element_located((By.TAG_NAME, "body"))
        )
        time.sleep(3)

        # Find all elements containing a bullet point • (expert
        expert_elements = driver.find_elements(By.XPATH, "//*[contains(
        # Looks for any element whose text contains the bullet point

        print(f"Expert elements found: {len(expert_elements)}")

        if expert_elements:
            print(f"Clicking first expert: {expert_elements[0].text}")
            expert_elements[0].click()
            # Clicks the first expert name to see what happens
            time.sleep(3)
            # Waits for any popup or side panel to load

            # Capture full page content after click
            content = driver.find_element(By.TAG_NAME, "body").text
            print("\n--- PAGE CONTENT AFTER CLICK ---")
            print(content)

    except Exception as e:
        print(f"Error: {e}")
    finally:
        driver.quit()

inspect_expert_click(URL)
```

Share the output — this tells us whether clicking an expert opens a side panel with availability and screener responses, which will determine how we extract that data for all 39 experts.

Expert elements found: 3
0 Clicking first expert: • Adam Smith --- PAGE CONTENT AFTER CLICK --- Projects Calls Autobooking Surveys Sign In Integrat...

PASTED

output: ""

This is exactly what we needed. Clicking an expert expands their profile inline showing:

- Full work experience
- Screener questions and answers
- Availability status and rate

Now we can build the full extraction script:

python

```
# Cell 4 – Fetch all expert data by clicking each one
def fetch_all_experts(url: str) -> list:
    driver = create_driver()
    experts = []

    try:
        driver.get(url)
        WebDriverWait(driver, 10).until(
```

```

        EC.presence_of_element_located((By.TAG_NAME, "body"
    )
    time.sleep(3)

    # Click "Load more" until all experts are visible
    while True:
        try:
            load_more = driver.find_element(By.XPATH, "//button[
            load_more.click()
            time.sleep(2)
        except:
            break

    # Find all expert bullet point elements
    expert_elements = driver.find_elements(By.XPATH, "//*[contains(
    print(f"Total experts found: {len(expert_elements)}")

    for i, expert_el in enumerate(expert_elements):
        try:
            # Scroll expert into view and click to expand profile
            driver.execute_script("arguments[0].scrollIntoView();", expert_el)
            # Scrolls the page so the expert is visible before clicking
            time.sleep(1)
            expert_el.click()
            time.sleep(2)
            # Waits for the expanded profile to load

            # Capture full page text after click
            content = driver.find_element(By.TAG_NAME, "body").text
            lines = content.split('\n')

            # Find the expert's name (line after bullet point)
            name = expert_el.text.replace('•', '').strip()
            name = ''.join(c for c in name if c.isalnum() or c in ' _-.,:;!@#$%^&*(){}[]\|/`~"\'"
            # Removes special characters like stars ★ from the name

            # Find the index of the name in the page content

```

```

name_idx = next((i for i, l in enumerate(lines)
# Finds the line containing the expert's name in

if name_idx is None:
    continue

# Extract role/company (line before name)
role_company = lines[name_idx - 1].strip() if name_idx > 0 else None

# Extract geography, segment, rate, status
geo = lines[name_idx + 1].strip() if name_idx + 1 < len(lines) else None
segment = lines[name_idx + 2].strip() if name_idx + 2 < len(lines) else None
rate = lines[name_idx + 3].strip() if name_idx + 3 < len(lines) else None
status = lines[name_idx + 4].strip() if name_idx + 4 < len(lines) else None

# Extract screener Q&A (numbered questions start with a number)
screener = []
for line in lines[name_idx:]:
    if re.match(r'^\d+\.', line.strip()):
        screener.append(line.strip())
    # Captures any line starting with a number

experts.append({
    "name": name,
    "role_company": role_company,
    "geography": geo,
    "segment": segment,
    "rate": rate,
    "status": status,
    "screener_responses": screener
})

print(f"Extracted ({i+1}/{len(expert_elements)})")

except Exception as e:
    print(f"Error on expert {i+1}: {e}")
    continue

```



```
except Exception as e:
    print(f"Fatal error: {e}")
finally:
    driver.quit()

return experts
```

python

Cell 5 – Display results

```
def display_experts(experts: list) -> None:
    for e in experts:
        print("=" * 80)
        print(f"Name:           {e['name']}")
        print(f"Role:           {e['role_company']}")
        print(f"Geography:        {e['geography']}")
        print(f"Segment:          {e['segment']}")
        print(f"Rate:             {e['rate']}")
        print(f>Status:          {e['status']}")
        if e['screeener_responses']:
            print("Screeener Responses:")
            for q in e['screeener_responses']:
                print(f"    {q}")
        else:
            print("Screeener Responses: None")
    print("=" * 80)
    print(f"\nTotal experts extracted: {len(experts)}")
```

python

Cell 6 – Run everything

```
experts = fetch_all_experts(URL)
display_experts(experts)
```

What to expect when you run it:

1. Edge browser opens and loads the page
2. Clicks "Load more" to get all 39 experts
3. Clicks each expert one by one, pausing to let the profile expand
4. Extracts all data including screener responses
5. Closes the browser and prints a structured summary

Run it and share the output — it may need minor tweaks depending on how the screener answers align in the text.

```
Total experts found: 30 Error on expert 1: Message:
element click intercepted:
Element <div class="sc-d
cJsrY sc-ikkxIA jxRhKO k
SUVNR" style="gap: 0.3r...
```

PASTED

output: ""

Two errors are happening — both fixable:

1. **Element click intercepted** — the navbar is covering the expert elements when scrolling
2. **Stale element reference** — after clicking, the page re-renders and the old element references break

The fix is to re-find elements after each click and use JavaScript to click instead of a regular click:

python

```

# Cell 4 – Fixed fetch function
def fetch_all_experts(url: str) -> list:
    driver = create_driver()
    experts = []

    try:
        driver.get(url)
        WebDriverWait(driver, 10).until(
            EC.presence_of_element_located((By.TAG_NAME, "body"))
        )
        time.sleep(3)

        # Click "Load more" until all experts are visible
        while True:
            try:
                load_more = driver.find_element(By.XPATH, "//button")
                driver.execute_script("arguments[0].click();", load_more)
                # Uses JavaScript to click instead of Selenium
                time.sleep(2)
            except:
                break

        # Get total count of bullet point elements
        total = len(driver.find_elements(By.XPATH, "//*[contains(text, 'expert')]"))
        print(f"Total experts found: {total}")

        for i in range(total):
            try:
                # Re-find ALL bullet elements fresh each iteration
                # This prevents stale element errors after page load
                expert_elements = driver.find_elements(By.XPATH, "//*[contains(text, 'expert')]")

                if i >= len(expert_elements):
                    break

                expert_el = expert_elements[i]

```

```

name = expert_el.text.replace('•', '').strip()
name = ''.join(c for c in name if c.isalnum() or c.isspace())
# Cleans special characters like ★ from names

# Scroll element into view with offset to avoid
driver.execute_script(
    "arguments[0].scrollIntoView(true); window.scrollBy(0, 100);",
    expert_el
)
# Scrolls to element then nudges up 150px to clear
time.sleep(1)

# Use JavaScript click to bypass any overlay block
driver.execute_script("arguments[0].click();", expert_el)
time.sleep(2)

# Capture page content after expanding profile
content = driver.find_element(By.TAG_NAME, "body").text
lines = content.split('\n')

# Find expert name position in expanded content
clean_name = name.strip()
first_word = clean_name.split()[0]
if clean_name:
    name_idx = next(
        (idx for idx, l in enumerate(lines)
         if first_word in l and '•' not in l and len(l) > 10),
        None
    )

if name_idx is None:
    print(f"Could not locate {name} in page content")
    continue

# Extract fields relative to name position
role_company = lines[name_idx - 1].strip() if name_idx > 0 else None
geo = lines[name_idx + 1].strip() if name_idx + 1 < len(lines) else None
segment = lines[name_idx + 2].strip() if name_idx + 2 < len(lines) else None

```

```

        rate          = lines[name_idx + 3].strip() if name_idx + 3 < len(lines) else None
        status        = lines[name_idx + 4].strip() if name_idx + 4 < len(lines) else None

        # Extract screener Q&A – lines starting with a question mark
        screener = []
        in_screener = False
        for line in lines[name_idx:]:
            if re.match(r'^\d+\.', line.strip()):
                screener.append(line.strip())
                in_screener = True
            elif in_screener and line.strip() and not re.match(r'^\d+\.', line.strip()):
                screener.append("    -> " + line.strip())
            # Captures answer lines immediately following a question
            else:
                break

        experts.append({
            "name": name,
            "role_company": role_company,
            "geography": geo,
            "segment": segment,
            "rate": rate,
            "status": status,
            "screener_responses": screener
        })

        print(f"Extracted ({i+1}/{total}): {name}")

    except Exception as e:
        print(f"Error on expert {i+1}: {e}")
        continue

except Exception as e:
    print(f"Fatal error: {e}")
finally:
    driver.quit()

return experts

```

python

```
# Cell 5 – Display results (unchanged)
def display_experts(experts: list) -> None:
    for e in experts:
        print("=" * 80)
        print(f"Name:          {e['name']}")
        print(f"Role:          {e['role_company']}")
        print(f"Geography:       {e['geography']}")
        print(f"Segment:        {e['segment']}")
        print(f"Rate:          {e['rate']}")
        print(f"Status:         {e['status']}")
        if e['screener_responses']:
            print("Screener Responses:")
            for q in e['screener_responses']:
                print(f"    {q}")
        else:
            print("Screener Responses: None")
    print("=" * 80)
    print(f"\nTotal experts extracted: {len(experts)}")
```

python

```
# Cell 6 – Run
experts = fetch_all_experts(URL)
display_experts(experts)
```

Key fixes made:

- Re-finds all expert elements fresh on every loop iteration — fixes stale element errors
- Uses `execute_script("arguments[0].click()")` — bypasses navbar blocking the click

- Scrolls up 150px after positioning — clears the fixed top navbar
- Captures answer lines directly after each question number

The whole page has loaded and all 30 experts have been found. how do i now obtain the following for each expert: geography, segment, role, company, screener responses, and availability?

The page is loaded and ready. Now we need to click each expert one by one and capture their expanded profile. Replace Cell 4 with this:

python

Cell 4 – Click each expert and extract data

```
def fetch_all_experts(url: str) -> list:
    driver = create_driver()
    experts = []

    try:
        driver.get(url)
        WebDriverWait(driver, 10).until(
            EC.presence_of_element_located((By.TAG_NAME, "body"))
        )
        time.sleep(3)

        # Click "Load more" until all experts are visible
        while True:
            try:
                load_more = driver.find_element(By.XPATH, "//button[contains(text, 'Load more')]")
                driver.execute_script("arguments[0].click();", load_more)
                time.sleep(2)
            except:
                break
```

```

total = len(driver.find_elements(By.XPATH, "//*[contains
print(f"Total experts found: {total}")

for i in range(total):
    try:
        # Re-find elements fresh each time to avoid stal
        expert_elements = driver.find_elements(By.XPATH
        if i >= len(expert_elements):
            break

        expert_el = expert_elements[i]

        # Scroll into view and clear navbar
        driver.execute_script(
            "arguments[0].scrollIntoView(true); window.s
            expert_el
        )
        time.sleep(1)

        # JavaScript click to bypass any overlay
        driver.execute_script("arguments[0].click();", e
        time.sleep(2)

        # Capture full page text after profile expands
        lines = driver.find_element(By.TAG_NAME, "body")
        lines = [l.strip() for l in lines if l.strip()]
        # Removes all blank lines for cleaner parsing

        # Find the expanded expert block – identified by
        # When expanded, name appears WITHOUT the bullet
        name_raw = expert_el.text.replace('•', '').strip
        name_clean = ''.join(c for c in name_raw if c.is
        first_word = name_clean.split()[0] if name_clean

        # Find name line in expanded content (no bullet
        name_idx = next(

```



```

        (idx for idx, l in enumerate(lines)
         if first_word in l
         and '•' not in l
         and len(l) < 80
         and idx > 10),
        # idx > 10 skips the navbar area at top of p
        None
    )

if name_idx is None:
    print(f"    Skipping {name_clean} – could not
    continue

# --- Extract fields ---

# Role and company: line immediately before the
role_company = lines[name_idx - 1] if name_idx > 0 else None

# Parse role and company from format:
# "CompanyName - Role Title (date)"
if ' - ' in role_company:
    company = role_company.split(' - ')[0].strip()
    role = ' - '.join(role_company.split(' - ')[1:])
    # Splits on first dash only, keeps rest as is
else:
    company = role_company
    role = ""

# Geography and segment: lines after name
geo      = lines[name_idx + 1] if name_idx + 1 < len(lines) else None
segment  = lines[name_idx + 2] if name_idx + 2 < len(lines) else None
rate     = lines[name_idx + 3] if name_idx + 3 < len(lines) else None
status   = lines[name_idx + 4] if name_idx + 4 < len(lines) else None

# Screener: numbered lines e.g. "1. Question" for
screener = []
j = name_idx

```

```
while j < len(lines):
    line = lines[j]
    if re.match(r'^\d+\.', line):
        question = line
        answer = lines[j + 1] if j + 1 < len(lines) else ''
        screener.append({"question": question, "answer": answer})
        # Pairs each numbered question with the next line
        j += 2
    elif line in ["Show work experience", "Contact us", "About us", "Home"]:
        break
    # Stops at action buttons – marks end of page
else:
    j += 1

experts.append({
    "name": name_clean,
    "company": company,
    "role": role,
    "geography": geo,
    "segment": segment,
    "rate": rate,
    "availability": status,
    "screener_responses": screener
})

print(f"Extracted ({i+1}/{total}): {name_clean}")

except Exception as e:
    print(f"Error on expert {i+1}: {e}")
    continue

except Exception as e:
    print(f"Fatal error: {e}")
finally:
    driver.quit()
```

```
return experts
```

python

Cell 5 – Display results

```
def display_experts(experts: list) -> None:
    for e in experts:
        print("=" * 80)
        print(f"Name:           {e['name']}")
        print(f"Company:          {e['company']}")
        print(f"Role:             {e['role']}")
        print(f"Geography:         {e['geography']}")
        print(f"Segment:           {e['segment']}")
        print(f"Rate:              {e['rate']}")
        print(f"Availability: {e['availability']}")
        if e['screeener_responses']:
            print("Screeener Responses:")
            for qa in e['screeener_responses']:
                print(f"  Q: {qa['question']}")
                print(f"  A: {qa['answer']}")
        else:
            print("Screeener Responses: None")
    print("=" * 80)
    print(f"\nTotal experts extracted: {len(experts)}")
```

python

Cell 6 – Run

```
experts = fetch_all_experts(URL)
display_experts(experts)
```

Three key improvements from before:

- **Company and role are now split** — `"Bellrock Group - Managing Director"` becomes `company = "Bellrock Group"` and `role = "Managing Director"`
- **Screener answers are paired with questions** — each Q&A is stored as a dictionary pair instead of a flat list
- **Stops cleanly** at action buttons like "Schedule a call" so it doesn't accidentally capture button text as screener content

Run it and share the output!

it's no longer working and i got an error when running (even though the page opened correctly on my screen):
 "Total experts found: 30 Error on expert 1: Message: stale element reference: stale element not found in the current frame (Session info: MicrosoftEdge=148.0.3967.54); For documentation on this error, please visit:
<https://www.selenium.dev/documentation/webdriver/troubleshooting/#stale-element-reference>

Show more

The stale element error is happening because the page re-renders after each click, invalidating the old element references even though we're re-finding them. The fix is to extract all expert names first, then click each one by searching for it fresh each time:

python

```
# Cell 4 – Fixed: extract names first, then click each one fresh
def fetch_all_experts(url: str) -> list:
    driver = create_driver()
    experts = []
```

```

try:
    driver.get(url)
    WebDriverWait(driver, 10).until(
        EC.presence_of_element_located((By.TAG_NAME, "body"))
    )
    time.sleep(3)

    # Click "Load more" until all experts are visible
    while True:
        try:
            load_more = driver.find_element(By.XPATH, "//button")
            driver.execute_script("arguments[0].click();", load_more)
            time.sleep(2)
        except:
            break

    # Step 1 – Extract all expert names ONCE before any clicks
    # This avoids stale references caused by page re-renders
    all_bullet_elements = driver.find_elements(By.XPATH, "//div")
    expert_names = []
    for el in all_bullet_elements:
        raw = el.text.replace('•', '').strip()
        clean = ''.join(c for c in raw if c.isalnum() or c == ' ')
        if clean:
            expert_names.append(clean)

    print(f"Total experts found: {len(expert_names)}")

    # Step 2 – Loop through names, find each element fresh,
    for i, name in enumerate(expert_names):
        try:
            first_word = name.split()[0]

            # Find the bullet element for this expert fresh
            # Searches by first word of name to avoid stale
            expert_el = driver.find_element(

```

```

        By.XPATH,
        f"//*[contains(text(), '•') and contains(text(),
    )

# Scroll into view and clear fixed navbar
driver.execute_script(
    "arguments[0].scrollIntoView(true); window.scrollTo(0, 0);",
    expert_el
)
time.sleep(1)

# JavaScript click bypasses any overlay blocking
driver.execute_script("arguments[0].click();", expert_el)
time.sleep(2)

# Capture and clean page text after profile expanded
lines = driver.find_element(By.TAG_NAME, "body").text.splitlines()
lines = [l.strip() for l in lines if l.strip()]

# Find expanded name line – no bullet, short, and not a link
name_idx = next(
    (idx for idx, l in enumerate(lines)
     if first_word in l
     and '•' not in l
     and len(l) < 80
     and idx > 10),
    None
)

if name_idx is None:
    print(f"Skipping {name} – could not locate name line")
    continue

# Extract role/company from line before name
role_company = lines[name_idx - 1] if name_idx > 0 else ""
if '-' in role_company:
    company = role_company.split('-')[0].strip()

```

```

        role = ' - '.join(role_company.split(' - '))
    else:
        company = role_company
        role = ""

# Extract geo, segment, rate, availability from
geo          = lines[name_idx + 1] if name_idx < len(lines) - 1 else ""
segment      = lines[name_idx + 2] if name_idx < len(lines) - 2 else ""
rate         = lines[name_idx + 3] if name_idx < len(lines) - 3 else ""
availability = lines[name_idx + 4] if name_idx < len(lines) - 4 else ""

# Extract screener Q&A pairs
screener = []
j = name_idx
stop_words = ["Show work experience", "Contact expert", "Schedule a call", "Contact expert", "Schedule a call"]
while j < len(lines):
    if re.match(r'^\d+\.', lines[j]):
        question = lines[j]
        answer = lines[j + 1] if j + 1 < len(lines) else ""
        screener.append({"question": question, "answer": answer})
        j += 2
    elif lines[j] in stop_words:
        break
    else:
        j += 1

experts.append({
    "name": name,
    "company": company,
    "role": role,
    "geography": geo,
    "segment": segment,
    "rate": rate,
    "availability": availability,
    "screener_responses": screener
})

```

```

        print(f"Extracted ({i+1}/{len(expert_names)}):

    except Exception as e:
        print(f"Error on expert {i+1} ({name}): {e}")
        continue

except Exception as e:
    print(f"Fatal error: {e}")
finally:
    driver.quit()

return experts

```

python

Cell 5 – Display results

```

def display_experts(experts: list) -> None:
    for e in experts:
        print("=" * 80)
        print(f"Name:          {e['name']}")
        print(f"Company:         {e['company']}")
        print(f"Role:            {e['role']}")
        print(f"Geography:       {e['geography']}")
        print(f"Segment:         {e['segment']}")
        print(f"Rate:            {e['rate']}")
        print(f"Availability:    {e['availability']}")
        if e['screener_responses']:
            print("Screener Responses:")
            for qa in e['screener_responses']:
                print(f"  Q: {qa['question']}")
                print(f"  A: {qa['answer']}")
        else:
            print("Screener Responses: None")

```



```
print("=" * 80)
print(f"\nTotal experts extracted: {len(experts)}")
```

python

```
# Cell 6 – Run
experts = fetch_all_experts(URL)
display_experts(experts)
```

What changed and why:

- **Extract all names first** — before any clicking happens, all 30 names are saved into a plain Python list. This list never goes stale because it's just strings, not browser elements.
- **Find each expert fresh by name** — instead of holding onto element references, the code searches for each expert by their first name at click time, getting a brand new reference each time.
- **No more stale errors** — since we never hold onto old element references across page re-renders, the stale element problem is eliminated entirely.

i get a consistent error - "name 're' is not defined"

Simple fix — `re` is missing from your imports. Update Cell 1 to include it:

python

```

# Cell 1 – Imports
import os
os.environ['WDM_SSL_VERIFY'] = '0'

import re
# Regular expressions library – used to detect numbered screeners

from selenium import webdriver
from selenium.webdriver.edge.service import Service
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from webdriver_manager.microsoft import EdgeChromiumDriverManager
import time

```

Re-run Cell 1 first, then re-run Cell 6. That should fix it.

i would now like to view it in table, with a new row for each expert; columns as name, geography, segment, role, company, screener responses, and availability

Add this new cell after Cell 5:

```

python

# Cell 7 – Display results as a table
import pandas as pd
# Pandas library for creating and displaying structured tables

def display_as_table(experts: list):
    rows = []
    for e in experts:

```

```

# Format screener Q&A into readable multiline text for
if e['screener_responses']:
    screener_text = "\n".join(
        [f"Q: {qa['question']}\nA: {qa['answer']}" for qa in e['screener_responses']]
    )
else:
    screener_text = "None"

rows.append({
    "Name": e['name'],
    "Geography": e['geography'],
    "Segment": e['segment'],
    "Role": e['role'],
    "Company": e['company'],
    "Screener Responses": screener_text,
    "Availability": e['availability']
})

df = pd.DataFrame(rows)
# Creates a structured table from the list of dictionaries

# Display settings – shows full text without truncation
pd.set_option('display.max_colwidth', None)
pd.set_option('display.max_rows', None)

return df

df = display_as_table(experts)
df
# In Jupyter, ending a cell with a variable name displays it as

```

If `pandas` isn't installed, run this first in a separate cell:

```
python
```

```
# Install pandas if not already installed
import subprocess
subprocess.run(["pip", "install", "pandas"])
```

Then re-run Cell 7. The table will render inline in your Jupyter notebook with each expert as a row and all columns visible.

the 'segment' column isn't parsing correctly - it's returning the first screener question instead

This is a parsing issue — after the name, the lines aren't always in the same order. Let's print the raw lines around a few experts to see exactly what the structure looks like:

python

```
# Cell 8 – Diagnostic: print raw lines around each expert name
def diagnose_parsing(url: str) -> None:
    driver = create_driver()
    try:
        driver.get(url)
        WebDriverWait(driver, 10).until(
            EC.presence_of_element_located((By.TAG_NAME, "body"))
        )
        time.sleep(3)

        # Click first expert only for inspection
        expert_el = driver.find_element(By.XPATH, "//*[contains(
            driver.execute_script(
                "arguments[0].scrollIntoView(true); window.scrollTo(
                    0, document.querySelector(arguments[0]).offsetTop - 100;
            ), expert_el
        )])")
```

```

time.sleep(1)
driver.execute_script("arguments[0].click();", expert_el)
time.sleep(2)

lines = driver.find_element(By.TAG_NAME, "body").text.splitlines()
lines = [l.strip() for l in lines if l.strip()]

# Find name and print 10 lines before and after it
name_raw = expert_el.text.replace('•', '').strip()
first_word = name_raw.split()[0]
name_idx = next(
    (idx for idx, l in enumerate(lines)
     if first_word in l and '•' not in l and len(l) < 80),
    None
)

if name_idx:
    print(f"Name found at line {name_idx}")
    print("\n--- Lines around expert name ---")
    for idx in range(max(0, name_idx - 2), min(len(lines), name_idx + 2)):
        print(f"Line {idx}: {lines[idx]}")

except Exception as e:
    print(f"Error: {e}")
finally:
    driver.quit()

diagnose_parsing(URL)

```

Run this and share the output — it will show us the exact line order around the expert name so we can fix the segment, geography, and availability parsing precisely.

i keep getting the following error - got this from running cell 8 "Error: Message: stale element reference: stale element not found in the current frame (Session info: MicrosoftEdge=148.0.3967.54); For documentation on this error, please visit:

<https://www.selenium.dev/documentation/webdriver/troubleshooting/errors#staleelementreferenceexception>

Stacktrace: msedgedriver!GetHandleVerifier

[0x7ff68a7de295+e0f5]

msedgedriver!GetHandleVerifier

[0x7ff68a7de2f4+e154]..."

The stale error happens because we're holding onto the element reference after the click. Let's fix by grabbing the name text BEFORE clicking:

python

```
# Cell 8 – Diagnostic: grab name first, then click, then inspect
```

```
def diagnose_parsing(url: str) -> None:
```

```
    driver = create_driver()
```

```
    try:
```

```
        driver.get(url)
```

```
        WebDriverWait(driver, 10).until(
```

```
            EC.presence_of_element_located((By.TAG_NAME, "body"))
```

```
        )
```

```
        time.sleep(3)
```

```
    # Step 1 – Get name text BEFORE clicking (avoids stale
```

```
    expert_el = driver.find_element(By.XPATH, "//*[contains
```

```
    name_raw = expert_el.text.replace('•', '').strip()
```

```
    first_word = name_raw.split()[0]
```

```
    print(f"Clicking expert: {name_raw}")
```

```
    # Step 2 – Scroll and click
```

```
    driver.execute_script(
```

```

        "arguments[0].scrollIntoView(true); window.scrollTo(0, 0);
        expert_el
    )
    time.sleep(1)
    driver.execute_script("arguments[0].click();", expert_el)
    time.sleep(2)

    # Step 3 – Now get fresh page content AFTER click
    # expert_el is intentionally not used again after this
    lines = driver.find_element(By.TAG_NAME, "body").text.splitlines()
    lines = [l.strip() for l in lines if l.strip()]

    # Step 4 – Find name in expanded content
    name_idx = next(
        (idx for idx, l in enumerate(lines)
         if first_word in l and '•' not in l and len(l) < 80),
        None
    )

    if name_idx:
        print(f"\nName '{name_raw}' found at line {name_idx}")
        print("\n--- Raw lines around expert (lines -2 to +2):")
        for idx in range(max(0, name_idx - 2), min(len(lines), name_idx + 2)):
            print(f"  Line {idx:3}: {lines[idx]}")
    else:
        print(f"Could not find '{name_raw}' in page content")
        print("\n--- First 50 lines of page for inspection:")
        for idx, l in enumerate(lines[:50]):
            print(f"  Line {idx:3}: {l}")

except Exception as e:
    print(f"Error: {e}")
finally:
    driver.quit()

diagnose_parsing(URL)

```

What changed: the name text is saved into a plain string variable `name_raw` before the click happens. After the click the page re-renders and `expert_el` goes stale — but we never touch it again, so no stale error. Run this and share the output.

```
output: "Clicking expert: Adam Smith Name 'Adam Smith'
found at line 65 --- Raw lines around expert (lines -2 to
+15) --- Line 63: Status Line 64: Bellrock Group -
Current Managing Director - Technologies (Oct 2024 -
Present) Line 65: Adam Smith Line 66: Current Chief
Technology Officer at Bellrock Property & Facilities
Management (Jan 2023 - Present) Current Managing
```

Show more

Now we can see the exact structure. The expanded profile has **no geography or segment lines directly after the name** — instead it shows full work experience and screener questions. Geography and segment are actually in the **collapsed list view** (before clicking).

The fix is to extract geography, segment, and availability from the page **before** clicking, then extract role, company, and screener **after** clicking:

```
python
```

```
# Cell 4 – Fixed: extract geo/segment before click, screener af
def fetch_all_experts(url: str) -> list:
    driver = create_driver()
```



```

experts = []

try:
    driver.get(url)
    WebDriverWait(driver, 10).until(
        EC.presence_of_element_located((By.TAG_NAME, "body")
    )
    time.sleep(3)

    # Click "Load more" until all experts are visible
    while True:
        try:
            load_more = driver.find_element(By.XPATH, "//button")
            driver.execute_script("arguments[0].click();", load_more)
            time.sleep(2)
        except:
            break

    # Step 1 – Extract all data from collapsed list BEFORE clicking
    # At this point geo, segment, rate, availability are all visible
    pre_click_lines = driver.find_element(By.TAG_NAME, "body").text.split("\n")
    pre_click_lines = [l.strip() for l in pre_click_lines if l.strip()]

    # Build a lookup of name -> {geo, segment, rate, availability}
    # Structure in list view: role_company / • Name / geo / segment / rate / availability
    pre_click_data = {}
    for idx, line in enumerate(pre_click_lines):
        if line.startswith('•'):
            name = line.replace('•', '').strip()
            name = ''.join(c for c in name if c.isalnum() or c in ' / ')
            geo = pre_click_lines[idx + 1] if idx + 1 < len(pre_click_lines) else ''
            segment = pre_click_lines[idx + 2] if idx + 2 < len(pre_click_lines) else ''
            rate = pre_click_lines[idx + 3] if idx + 3 < len(pre_click_lines) else ''
            availability = pre_click_lines[idx + 4] if idx + 4 < len(pre_click_lines) else ''
            pre_click_data[name] = {
                "geography": geo,
                "segment": segment,

```

```

        "rate": rate,
        "availability": availability
    }

print(f"Total experts found: {len(pre_click_data)}")

# Step 2 – Click each expert to get role, company, and s
for i, name in enumerate(pre_click_data.keys()):
    try:
        first_word = name.split()[0]

        # Find bullet element fresh each time by expert
        expert_el = driver.find_element(
            By.XPATH,
            f"//*[contains(text(), '•') and contains(text(), '{first_word}')]")

        # Save name text before click to avoid stale re
        driver.execute_script(
            "arguments[0].scrollIntoView(true); window.s
            expert_el
        )
        time.sleep(1)
        driver.execute_script("arguments[0].click();", expert_el)
        time.sleep(2)

        # Get fresh page content after profile expands
        post_lines = driver.find_element(By.TAG_NAME, "div").text.split("\n")
        post_lines = [l.strip() for l in post_lines if l.strip()]

        # Find name in expanded content (no bullet, show name)
        name_idx = next(
            (idx for idx, l in enumerate(post_lines)
             if first_word in l and '•' not in l
             and len(l) < 80 and idx > 10),
            None
        )

```

```

if name_idx is None:
    print(f"    Skipping {name} – could not locate
    continue

# Role and company: line immediately before name
role_company = post_lines[name_idx - 1] if name_
if ' - ' in role_company:
    company = role_company.split(' - ')[0].strip
    role     = ' - '.join(role_company.split(' -
else:
    company = role_company
    role     = ""

# Screener: starts at line after work experience
# Skip line 66 (full work experience) and start
screener = []
stop_words = ["Show work experience", "Contact p
            "Contact expert", "Schedule a call
            "Add to shortlist", "Not intereste

j = name_idx + 2
# Starts at +2 to skip the long work experience
while j < len(post_lines):
    if re.match(r'^\d+\.', post_lines[j]):
        question = post_lines[j]
        answer = post_lines[j + 1] if (
            j + 1 < len(post_lines) and
            not re.match(r'^\d+\.', post_lines[j
            post_lines[j + 1] not in stop_words
        ) else "No answer"
        screener.append({"question": question,
            j += 2
    elif post_lines[j] in stop_words:
        break
    else:
        j += 1

```

```

# Merge pre-click data with post-click data
entry = pre_click_data[name]
experts.append({
    "name": name,
    "company": company,
    "role": role,
    "geography": entry["geography"],
    "segment": entry["segment"],
    "rate": entry["rate"],
    "availability": entry["availability"],
    "screener_responses": screener
})

print(f"Extracted ({i+1}/{len(pre_click_data)})")

except Exception as e:
    print(f"Error on expert {i+1} ({name}): {e}")
    continue

except Exception as e:
    print(f"Fatal error: {e}")
finally:
    driver.quit()

return experts

```

python

```

# Cell 5 – Display as table
def display_as_table(experts: list):
    rows = []
    for e in experts:
        screener_text = "\n".join(
            [f"Q: {qa['question']}\nA: {qa['answer']}" for qa in e['screener_responses']]
        ) if e['screener_responses'] else "None"

```

```
rows.append({
    "Name": e['name'],
    "Geography": e['geography'],
    "Segment": e['segment'],
    "Role": e['role'],
    "Company": e['company'],
    "Screener Responses": screener_text,
    "Availability": e['availability']
})

df = pd.DataFrame(rows)
pd.set_option('display.max_colwidth', None)
pd.set_option('display.max_rows', None)
return df
```

python

```
# Cell 6 – Run
experts = fetch_all_experts(URL)
df = display_as_table(experts)
df
```

Key fix: geography, segment, rate, and availability are now read from the **collapsed list view before any clicking** — exactly where we know they appear correctly. Role, company, and screener are read from the **expanded profile after clicking**. The two are merged at the end.

